

# Developer Testing

*Tests are the safety net that lets you change code without fear.*

# Tests Buy You Courage

Every principle so far assumed you can change code safely. Tests are what make that true — they turn 'I hope' into 'I know'.

## Confidence

Refactor and extend without breaking what worked.

## Feedback

Catch mistakes in seconds, not in production.

## Design pressure

Hard-to-test code is usually badly designed.

**The other half of defending correctness:** defensive programming stops bad values at the barricade; tests prove the code behind it is actually right.

# Learning Objectives

- Write clear unit tests with the Arrange–Act–Assert shape.
- Use test doubles to isolate the unit under test.
- Know when to reach for integration tests instead.
- See why testable code and good design are the same thing.

# Bugs and Regressions

Two different failures — and your tests catch them at two different moments.

## A bug

Behavior that never matched the requirement — wrong the first time anyone ran it. A new test catches it the first time it runs.

## A regression

Behavior that used to be correct and broke — usually because of a change made somewhere else. An old test catches it every time it runs after.

*regression* — Latin regredi: re- (back) + gradi (to step). The code has literally stepped backwards.

**A test earns its keep the second time it runs.**

# Turn Every Bug Into a Test

A bug report is a missing test. Reproduce it before you fix it — then keep the test forever.

Java

*the off-by-one that shipped: orders of exactly 100 got no discount*

```
// 1 · RED – reproduce the report as a failing test
@Test void appliesDiscountAtExactlyOneHundred() {
    assertEquals(90.0, order(100.0).total(), 0.001);    // fails: got 100.0
}

// 2 · GREEN – fix the boundary
-   return total > 100 ? 0.10 : 0.0;
+   return total >= 100 ? 0.10 : 0.0;

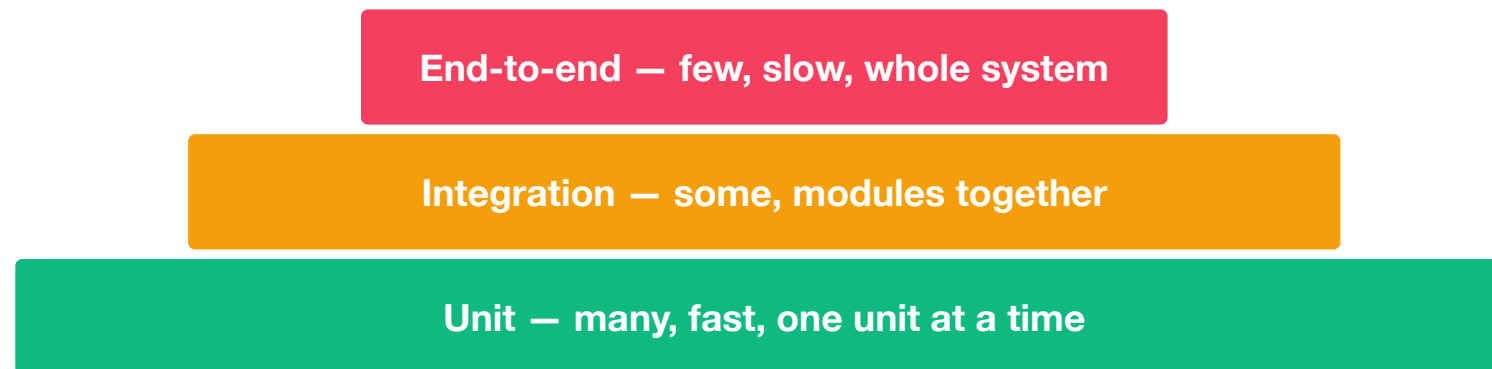
// 3 · KEEP – it stays in the suite; from now on it is a regression test
```

**Never fix a bug without a failing test that proves it.** Fix it without one and nothing stops it coming back.

*For legacy code you don't understand yet, characterization tests pin down what it does today (Feathers).*

# The Test Pyramid

Many fast, focused tests at the bottom; a few slow, broad ones at the top.



Push tests down the pyramid: cheaper to write, faster to run, easier to trust.

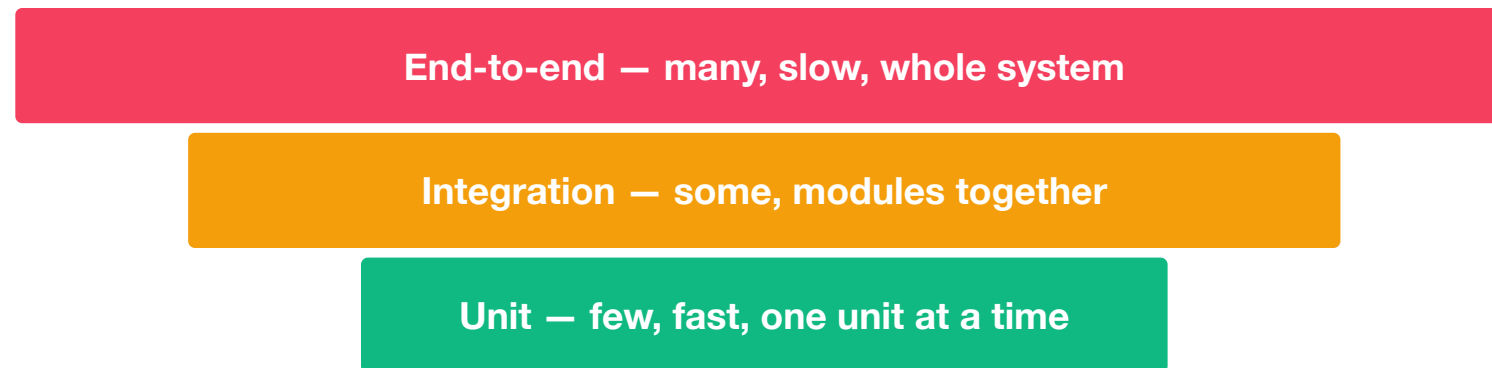
SECTION 01

# Unit Testing

*Test one small unit in isolation — fast and focused.*

# The Ice-Cream Cone

In practice many teams end up here — the pyramid flipped upside down: many slow end-to-end tests, barely any unit tests.



- Slow feedback — a full run takes minutes, so it runs only in CI, hours later.
- Flaky — timing, network and shared data turn green red at random.
- Failures don't localize — a red end-to-end test says 'something broke', not where.

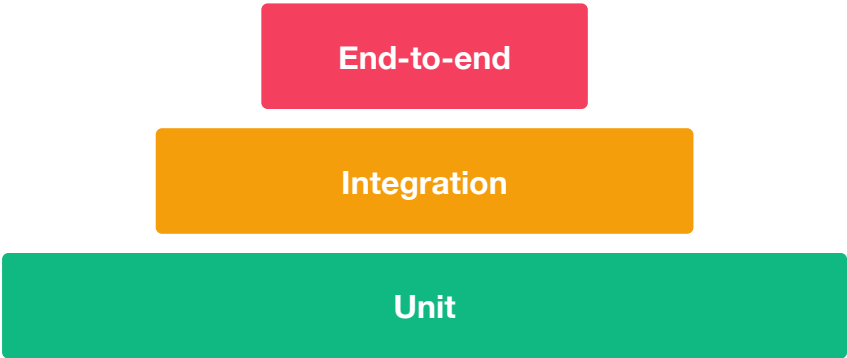
**The fix isn't more end-to-end tests — it's pushing behavior down into fast unit tests.**



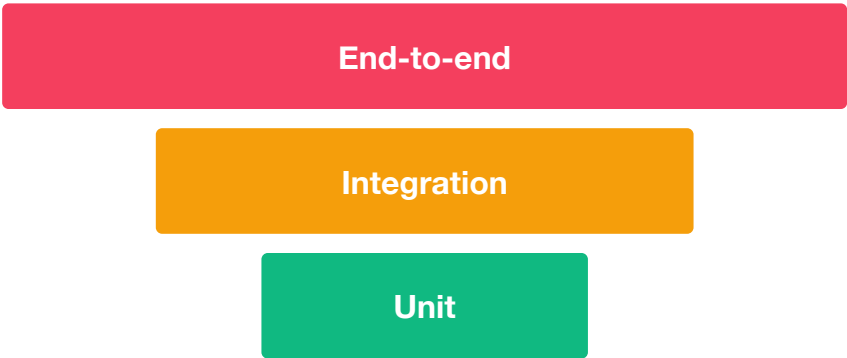
# Two Shapes, Same Tests

The same test budget, spent two ways. The shape decides how fast you learn you broke something — and how precisely.

PYRAMID — what you want



ICE-CREAM CONE — what you get



|                                                 | Pyramid                                                                                                                                     | Ice-cream cone                  |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Feedback                                        | seconds, on your machine                                                                                                                    | minutes, in CI, hours later     |
| A red test says<br>CI — continuous integration: | this behavior broke<br><i>the build server that runs the whole suite automatically after every push, instead of you running it locally.</i> | something, somewhere broke      |
| Flakiness                                       | rare — no I/O to misbehave                                                                                                                  | routine — timing, network, data |

# Arrange · Act · Assert

Every unit test tells a tiny story in three beats — the AAA pattern:

## Arrange

*build the object and its inputs*

## Act

*call the one behavior under test*

## Assert

*check the outcome — and nothing else*

### Java / JUnit

```
@Test
void addsTwoNumbers() {
    var calc = new Calc(); // arrange
    int r = calc.add(2, 3); // act
    assertEquals(5, r);    // assert
}
```

### C# / xUnit

```
[Fact]
public void AddsTwoNumbers() {
    var calc = new Calc(); // arrange
    var r = calc.Add(2, 3); // act
    Assert.Equal(5, r);    // assert
}
```

### Python / pytest

```
def test_adds_two_numbers():
    calc = Calc()           # arrange
    r = calc.add(2, 3)       # act
    assert r == 5           # assert
```

# What Makes a Good Test Suite

A suite is more than a pile of tests. Three properties separate one that pays for itself from one that becomes a burden:

## It is integrated into the development cycle

*You only get value from tests you actively use. A suite that runs nightly — or only before a release — has already stopped paying you back.*

## It targets the parts that matter

*Not all production code deserves equal attention. Cover the heart of the application — its domain model — before glue code, getters and configuration.*

## It gives maximum value for minimum maintenance

*Every test is code you must keep working. A suite that costs more than it saves gets ignored first, then deleted.*

*“Program testing can be used to show the presence of bugs, but never to show their absence.” — Edsger W. Dijkstra*

# Four Attributes of a Good Unit Test

A unit test exercises one unit with no database, network or file system. Judge every one of them against four attributes (Khorikov):

## Protection against regressions

*Does it actually catch a break? The more meaningful production code a test exercises, the more it protects.*

## Resistance to refactoring

*Does it stay green when you improve the design without changing behavior? A test tied to implementation cries wolf.*

## Fast feedback

*How soon does it tell you? Slow tests get run less often, and late feedback costs far more to act on.*

## Maintainability

*How hard is it to read, and to keep running? An unreadable test is a future deletion.*

**The first three pull against each other — you can maximize any two, never all three.** Maintainability is the one you never trade away.

# What Is a ‘Unit’, Really?

‘One class’ is the easy answer. The useful one is a unit of behavior — a use case that may span a few classes. And ‘isolation’ means two different things:

|               | Classicist — Kent Beck           | Mockist — London · GOOS             |
|---------------|----------------------------------|-------------------------------------|
| A ‘unit’ is   | a behavior (may span classes)    | usually a single class              |
| ‘Isolation’   | tests isolated from each other   | object isolated from collaborators  |
| Collaborators | real; fake only I/O at the edges | mocked — verify the interactions    |
| Mocks         | sparing, at process boundaries   | central — “mock roles, not objects” |

**Prefer the unit of behavior:** tests that survive refactoring across classes and fail for one behavioral reason.

# Isolation Is an Architecture Decision

A behavior is easy to unit-test exactly when it depends on nothing but itself. Whether that's true is decided by your architecture, not your test framework.

**Rich domain · Hexagonal / DDD → unit = behavior**

*Rules live in dependency-free entities — call them and assert, no mocks. Mock only the ports at the edges (GOOS: “mock roles, not objects”).*

**Anemic · a service over a repository → unit = class**

*Logic sits in a service that calls a repository, so you must mock the repo to test it — London by necessity, and brittle when you refactor.*

**Tangled · logic woven into SQL / HTTP → no unit at all**

*No seam to isolate — only integration and end-to-end tests fit. This is how a codebase arrives at the ice-cream cone.*

**Refactoring course — see which ACME Orders behaviors need doubles, and why → [github.com/kaldiroglu/refactoring-course-student-materials](https://github.com/kaldiroglu/refactoring-course-student-materials)**

# The Same Rule, Three Architectures

One business rule — an order total — written three ways in C#. The architecture, not the test framework, decides what a unit test can reach.

## Rich domain · no doubles

```
// rule lives in the entity
public Money Total() =>
    lines.Aggregate(Money.Zero,
        (s,l) => s.Add(l.Subtotal()));

// test — nothing to mock
var o = new Order(Line(2, 10m));
Assert.Equal(20m, o.Total());
```

## Anemic · mock the repo

```
// logic in a service over a repo
public decimal Total(int id) {
    var o = _repo.Find(id);
    return o.Lines.Sum(l => l.Total);
}

// test — the repo must be doubled
var repo = new Mock<IOrderRepo>();
repo.Setup(r => r.Find(1))
    .Returns(order);
```

## Tangled · no seam

```
// rule buried in the query
var cmd = new SqlCommand(
    "SELECT SUM(qty*price) " +
    "FROM lines WHERE id=@id", conn);
var t = cmd.ExecuteScalar();

// no unit test reaches this —
// it needs a real database
```

**No double · one mock · no seam at all.** Same rule, same language — only the design changed. Testability is decided long before the first test is written.

# Test Doubles Isolate the Unit

To test a unit alone, replace its collaborators with stand-ins. This is why we depend on abstractions (DIP) — so we can swap in a fake.

## Common doubles

- Stub — returns canned answers
- Mock — verifies it was called right
- Fake — a lightweight working version

## Why it works

- The unit depends on an interface...
- ...so tests inject a double instead
- Design and testability go together



# A Method Worth Testing

Read it by color — the rules and the outside calls are woven together:

■ business rule ■ reaches outside — each one forces a double ■ plumbing

Java

*every cyan call is a collaborator the test must stand in for*

```
public User changePassword(String username, PasswordChangeRequest change) {
    User currentUser = userRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException("Username not found.));
    String currentPassword = currentUser.getPassword();
    String newPassword = encoder.encode(change.getNewPassword());
    TValidationException tex = new TValidationException("password");
    if (!isPasswordValid(change.getNewPassword()))
        tex.rejectNow("pattern", "not.valid");
    if (!encoder.matches(change.getOldPassword(), currentPassword))
        tex.rejectNow("old", "not.matches");
    if (encoder.matches(change.getNewPassword(), change.getOldPassword()))
        tex.rejectNow("old", "same");
    currentUser.setPassword(newPassword);
    return updateUser(currentUser);
}
```

*Amber and cyan interleave — that is the anemic tangle: you cannot reach a rule without going through a port.*

# Unit Test With Mocks

 mock setup act assert plumbing

## Java · JUnit 5 + Mockito

*eight cyan lines of setup — for one act and two asserts*

```
@Mock UserRepository userRepository;
@Mock PasswordEncoder encoder;
@InjectMocks UserService service;
@Test void changesPasswordWhenRequestIsValid() {
    User user = new User("ada", "ENC(old)"); // arrange
    var change = new PasswordChangeRequest("oldPass", "newPass1!");
    when(userRepository.findByUsername("ada")).thenReturn(Optional.of(user));
    when(encoder.encode("newPass1!")).thenReturn("ENC(new)");
    when(encoder.matches("oldPass", "ENC(old))).thenReturn(true);
    when(encoder.matches("newPass1!", "oldPass")).thenReturn(false);
    when(userRepository.save(user)).thenReturn(user);
    User result = service.changePassword("ada", change); // act
    assertThat(result.getPassword()).isEqualTo("ENC(new)"); // assert
    verify(userRepository).save(user);
}
```

**Classify it:** anemic — the middle column of page 15. Two mocks are needed because the rules live in the service, not in the model.

# Move the Rules, Lose the Mocks

The same feature, with the rules moved into the domain:

## rules in the domain

```
class User {  
    void changePassword(Password current,  
                        Password next,  
                        PasswordHasher hasher) {  
        if (!hasher.matches(current, hash))  
            throw new OldPasswordMismatch();  
        if (current.equals(next))  
            throw new SamePassword();  
        hash = hasher.hash(next);  
    }  
}  
// Password.of(raw) already rejected an  
// invalid pattern – see Deck 02
```

## its test — one fake, no mocks

```
@Test void rejectsWhenOldDoesNotMatch() {  
    var user = userWith("ENC(old)");  
    var hasher = new FakeHasher();  
    assertThrows(OldPasswordMismatch.class,  
        () -> user.changePassword(  
            Password.of("wrong1!"),  
            Password.of("newPass1!"), hasher));  
}  
  
// no @Mock · no when(...) · no verify(...)  
// the rule is tested where it lives
```

**Two mocks became one fake** — and the rejection path, where the real bug hides, is finally easy to test.

## SECTION 02

# Integration Testing

*Do the parts actually work together?*

# Test the Seams

Units can each pass while the wiring between them fails. Integration tests exercise real collaborators — a real database, a real HTTP call.

- Catch mismatches units can't: SQL, serialization, configuration.
- Slower and fewer — use them for the risky seams, not everything.
- Often run against a real DB in a container for fidelity.

# An Integration Test in C#

No doubles anywhere: the real routing, the real DI container, the real database — the whole pipeline,

exercised from the outside.

**C# · xUnit + WebApplicationFactory**

*one call exercises routing → model binding → DI → service → EF Core → SQL*

```
public class OrdersApiTests(WebApplicationFactory<Program> f)
    : IClassFixture<WebApplicationFactory<Program>> {
    private readonly HttpClient _client = f.CreateClient(); // the real app, in memory
    [Fact]
    public async Task PostOrder_CreatesItAndReturns201() {
        var response = await _client.PostAsJsonAsync("/orders",
            new { qty = 2, unitPrice = 10m });
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
        var created = await response.Content.ReadFromJsonAsync<OrderDto>();
        Assert.Equal(20m, created!.Total);
    }
}
```

**Catches what no unit test can:** routing, model binding, DI wiring, the JSON contract, SQL and transactions.

**Pays for it:** seconds instead of milliseconds, a database to keep alive, and a failure that says “somewhere in that chain”.

# Unit vs Integration

|               | Unit                  | Integration              |
|---------------|-----------------------|--------------------------|
| Scope         | One class / function  | Several parts together   |
| Collaborators | Replaced with doubles | Real (DB, queue, API)    |
| Speed         | Milliseconds          | Seconds                  |
| Count         | Many                  | Some                     |
| Catches       | Logic errors          | Wiring & contract errors |

# Red · Green · Refactor

Test-Driven Development flips the order: write a failing test first, then the code to pass it, then clean up. Tests become your design tool.



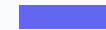
## Red

Write a small failing test for the next behavior.



## Green

Write the simplest code that makes it pass.



## Refactor

Improve the design — tests keep you safe.



# The Discipline: Three Laws

Kent Beck's loop, tightened into three rules by Robert C. Martin. They exist to keep each step small enough that you never lose control:

1

**Write no production code until a failing test demands it.**

*The test comes first — always. Otherwise you are writing code nothing asked for.*

2

**Write no more of a test than is enough to fail.**

*Not compiling counts as failing. Stop the moment it is red and go make it pass.*

3

**Write no more production code than is enough to pass that test.**

*The simplest thing that works. Generality waits until a test asks for it.*

**Each turn of the loop is seconds long.** If a step takes ten minutes, the step was too big.

*The by-product: you end up with only the code some test asked for — YAGNI, enforced by the loop rather than by willpower.*

# Test First or Test After?

The loop is the same either way. The order changes what the test is for — and whether you can trust it.

## Test after — code, then test

- Born green — you never watch it fail
- Describes the code, not the requirement
- Untestable design found too late
- “Later” tests that never arrive

## Test first — test, then code

- You watch it fail, so you know it can
- States the requirement, not the solution
- You are the first caller of your own API
- Done means green — scope stops growing

**A test that has never failed is not a safety net — it is a decoration.**

*Not a law: characterization tests for legacy code are written after on purpose (Feathers), and a spike can be thrown away and redone test-first.*

# Testable Means Well-Designed

If a unit is hard to test, that's a design smell — usually hidden dependencies or too many responsibilities.



## Hard to test →

- Tight coupling to concretes
- Doing too many things (low cohesion)
- Hidden state and side effects



## Easy to test →

- Depends on injected abstractions
- One responsibility, clear inputs/outputs
- Pure where possible

# AI Writes Tests — You Own Coverage

AI is excellent at generating test scaffolding and edge cases. But a test is only as good as the behavior it pins down.

- Ask for tests of edge cases: empty, null, boundary, error paths.
- Check that tests assert behavior — not just that code ran.
- Beware tests written to match buggy code; you decide what 'correct' is.

## REMEMBER THESE

# Key Takeaways

- ✓ Tests give you the courage to change code safely.
- ✓ Unit tests: one unit, isolated, fast — Arrange, Act, Assert.
- ✓ Test doubles isolate the unit; they rely on depending on abstractions.
- ✓ Integration tests check the seams with real collaborators.
- ✓ Hard to test usually means badly designed — testability is a design goal.

# Further Reading

The books behind this session. If you read only one, read the first.

 **Test Driven Development: By Example** Kent Beck · Addison-Wesley, 2002

*The red-green-refactor loop from its source, worked end to end.*

 **Growing Object-Oriented Software, Guided by Tests** Freeman & Pryce · Addison-Wesley, 2009

*How tests drive design — where mocks belong and what they are really for.*

 **Unit Testing: Principles, Practices, and Patterns** Vladimir Khorikov · Manning, 2020

*The modern synthesis: what makes a test valuable, and why most suites are brittle.*

 **Working Effectively with Legacy Code** Michael Feathers · Prentice Hall, 2004

*Seams and characterization tests — how to test code that was not built for testing.*

Fowler — “Mocks Aren’t Stubs”, the test-double taxonomy → [martinfowler.com/articles/mocksArentStubs.html](https://martinfowler.com/articles/mocksArentStubs.html)

# What's Next

MODULE 8 · SATURDAY 1 AUGUST

## Agentic Design & Coding

with Akin Kaldıroğlu

*Putting it all together: how to design and build software with an AI agent — while you stay the decision-maker.*

# Put a Safety Net Under It

*Before you change the ACME Orders code, pin its behavior with characterization tests — then refactor without fear.*

## Refactoring course · Characterization Tests

**Repo** [github.com/kaldiroglu/refactoring-course-student-materials](https://github.com/kaldiroglu/refactoring-course-student-materials)

### IN THIS SESSION, LOOK AT

- ▶ Exercises/Ex04-CharacterizationTests.md
- ▶ Slides-PDF/Ch06-SafetyNet.pdf
- ▶ .../src/test/... (the ACME Orders test suite)



---

# Questions?

*Test to move fast — safely.*